Python While Loops

در Python دو نوع حلقه اصلی داریم:

while

for

حلقه whileزمانی استفاده می‌شود که بخواهیم یک قطعه کد را تا زمانی که یک شرط درست است تکرار کنیم

ساختار while

ساختار کلی:

```
while condition:
  # code
```

یعنی تا زمانی که conditionبرابر Trueاست، کد داخل حلقه را اجرا کن

مثال

```
i = 1
while i < 6:
  print(i)
  i += 1
```

خروجی:

```
1
2
3
4
5
```

نکته بسیار مهم: تغییر متغیر

در مثال بالا داریم:

```
i += 1
```

اگر این خط را حذف کنیم:

```
i = 1
while i < 6:
  print(i)
```

مقدار **i**همیشه **1**باقی می‌ماند

بنابراین:

1 < 6 → True

1 < 6 → True

1 < 6 → True

...

و حلقه بی‌نهایت اجرا می‌شود به این حالت می‌گوییمInfinite Loop

شمارش معکوس با while

```
i = 5
while i > 0:
  print(i)
  i -= 1
```

خروجی:

5

4

3

2

1

چاپ اعداد زوج

```
i = 2
while i <= 10:
    print(i)
    i += 2
```

خروجی:

```
2
4
6
8
10
```

چاپ اعداد فرد

```
i = 1
while i <= 10:
    print(i)
    i += 2
```

خروجی:

```
1
3
5
7
9
```

استفاده از شرط if داخل while

می‌توانیم داخل حلقه از if نیز استفاده کنیم:

```
i = 1
while i <= 10:
    if i % 2 == 0:
        print(i)
    i += 1
```

خروجی:

```
2
4
6
8
10
```

break

دستور break باعث می‌شود حلقه فوراً متوقف شود؛ حتی اگر شرط while هنوز True باشد

مثال:

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

خروجی:

```
1
```

2

3

وقتی:

i == 3

برقرار شد، breakاجرا شده و حلقه تمام می‌شود

مثال break

فرض کنید می‌خواهیم اعداد را چاپ کنیم، اما وقتی به 7رسیدیم متوقف شویم:

```
i = 1
while i <= 10:
   print(i)
   if i == 7:
      break
   i += 1
```

خروجی:

1

2

3

4

5

6

7

continue **Continue :** باbreakمتفاوت است

Breakکل حلقه را متوقف می‌کند

Continueفقط **iteration فعلی** را متوقف می‌کند و به iteration بعدی می‌رود

مثال continue

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

خروجی:

```
1
2
4
5
6
```

چرا 3چاپ نشد؟

وقتی 3 == iبرقرار شد ، continue اجرا می‌شود در نتیجه ، print(i) در آن iteration اجرا نمی‌شود

نکته بسیار مهم درباره continue

در whileباید مراقب باشیم که تغییر متغیر قبل از continueانجام شود

مثلاً:

```
i = 0
while i < 6:
  i += 1
```

```
    if i == 3:
        continue
    print(i)
```

درست است اما این کد می‌تواند مشکل‌ساز شود:

```
i = 0
while i < 6:
    if i == 3:
        continue
    i += 1
```

وقتی i به 3 برسد، continue اجرا می‌شود و i += 1 دیگر اجرا نمی‌شود؛ بنابراین i همیشه 3 باقی می‌ماند و حلقه بی‌نهایت می‌شود

while ... else

در Python می‌توانیم بعد از while از else استفاده کنیم

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("Loop finished")
```

خروجی:

```
1
2
3
4
```

```
5
Loop finished
```

یعنی وقتی شرط i < 6 دیگر Trueنباشد، بخش elseاجرا می‌شود

break و else

یک نکته بسیار مهم اگر حلقه به وسیله breakمتوقف شود، elseاجرا نمی‌شود

مثلاً:

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
else:
  print("Loop finished")
```

خروجی:

```
1
2
3
```

تفاوت break و continue

دستور	عملکرد
break	کل حلقه را متوقف می‌کند
continue	فقط iteration فعلی را رد می‌کند
else	وقتی حلقه بدون breakتمام شود اجرا می‌شود

مثال:

```
# break
if i == 5:
    break
```

یعنی:

دیگر اصلاً ادامه نده

اما:

```
# continue
if i == 5:
    continue
```

یعنی این بار را رد کن و برو iteration بعدی

مثال– رمز عبور

فرض کنید کاربر باید رمز درست را وارد کند:

```
password = ""
while password != "1234":
    password = input("Enter password: ")
print("Access granted")
```

تا زمانی که کاربر رمز درست را وارد نکند، حلقه ادامه پیدا می‌کند

مثال– شمارش تلاش‌ها

```
password = ""
attempts = 0
while password != "1234" and attempts < 3:
    password = input("Enter password: ")
    attempts += 1
```

```
if password == "1234":
   print("Access granted")
else:
   print("Account locked")
```

مثال– جمع اعداد

برنامه‌ای بنویسیم که اعداد 1تا 5را جمع کند:

```
i = 1
total = 0
while i <= 5:
   total += i
   i += 1
print(total)
```

خروجی:

```
15
```

چون:

```
1 + 2 + 3 + 4 + 5 = 15
```

مثال– دریافت اعداد تا صفر

یکی از کاربردهای بسیار مهم whileزمانی است که نمی‌دانیم چند بار باید حلقه اجرا شود

```
number = int(input("Enter a number: "))
while number != 0:
   print(number)
   number = int(input("Enter a number: "))
print("Finished")
```

کاربر می‌تواند چند عدد وارد کند

وقتی:

0

وارد کند، حلقه تمام می‌شود

تمرین ۱

با استفاده از whileاعداد زیر را چاپ کنید:

1

2

3

4

5

6

7

8

9

10

تمرین ۲

با whileاعداد 10تا 1را به صورت معکوس چاپ کنید

خروجی:

10

9

8

...

1

تمرین ۳

با whileتمام اعداد زوج بین 1و 20را چاپ کنید

تمرین ۴

با whileتمام اعداد فرد بین 1و 20را چاپ کنید

تمرین ۵

برنامه‌ای بنویسید که اعداد 1تا 10را چاپ کند، اما وقتی به 5رسید، با استفاده از breakحلقه را متوقف کند

تمرین ۶

برنامه‌ای بنویسید که اعداد 1تا 10را چاپ کند، اما عدد 5را چاپ نکند

راهنمایی:

continue

تمرین ۷

برنامه‌ای بنویسید که مجموع اعداد 1تا 100را با استفاده از whileمحاسبه کند

تمرین ۸

از کاربر عدد دریافت کنید تا زمانی که کاربر 0وارد نکرده است، عدد را چاپ کنید

مثلاً:

Enter number: 5

5

Enter number: 8

8

Enter number: 12

12

Enter number: 0

Finished

تمرین ۹

برنامه‌ای بنویسید که یک عدد از کاربر دریافت کند و جدول ضرب آن را از 1تا 10نمایش دهد

مثلاً اگر کاربر وارد کند:

5

خروجی:

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

...

5 x 10 = 50

برنامه‌ای برای بازی حدس عدد بنویسید

عدد مخفی:

secret = 7

کاربر باید عدد را حدس بزند

اگر عدد کوچک‌تر بود:

Too low

اگر بزرگ‌تر بود:

Too high

اگر درست بود:

Correct!

و بازی تمام شود

راهنمایی:

secret = 7

guess = 0

```
while guess != secret:
    guess = int(input("Guess the number: "))
    if guess < secret:
        print("Too low")
    elif guess > secret:
        print("Too high")
    else:
        print("Correct!")
```

تمرین پروژه‌ای

یک سیستم ATM ساده با whileطراحی کنید

منو:

1. Check Balance
2. Deposit
3. Withdraw
4. Exit

مثلاً:

```
balance = 1000
```

برنامه باید تا زمانی که کاربر گزینه 4را انتخاب نکرده، منو را دوباره نمایش دهد در این پروژه دانش‌آموز می‌تواند از مفاهیم زیر به صورت همزمان استفاده کند:

while

if / elif / else

input()

int()

break